DynaGPU Project: Conversation Summary

Nov 7, 2026 · @shivam kumar

You set out to build a unique, serious open-source GPU project in the CNCF ecosystem. The conversation narrowed that to DynaGPU: weighted, capped and preemptible GPU time-sharing for Kubernetes pods, enforced by the GPU's own scheduler instead of by counting kernel launches. This summary covers every step, finding and decision so far.

Starting point and goal

You contribute to HAMi, a CNCF incubating project for sharing GPUs on Kubernetes, and you wanted to start your own open-source project in the GPU infrastructure layer: scheduling, enforcing limits and dynamic allocation.

The rules you set along the way:

- **Unique:** it must not exist anywhere, in open source or in CNCF, even today.
- **Serious,** not a hobby project.
- **Virtualization level,** like HAMi-core, not kernel level.
- **One domain to go deep in,** not a list of features.
- **Validated by real-world evidence** from issue trackers and companies running GPUs, not just claims that a problem is big.
- **Away from crowded areas** such as dynamic allocation and memory limits.
- **Kubernetes-native.** Later you chose to build it on DRA rather than on HAMi or KAI Scheduler.

The goal you settled on, in your own prompt: weighted, capped, preemptible GPU time-sharing between pods, where each pod keeps its own CUDA context, limits hold with CUDA graphs and long kernels, and there is no per-kernel-launch overhead.

How the idea evolved

The idea moved from a broad GPU QoS control plane to one narrow, validated mechanism: enforcing GPU time shares with the GPU's own scheduler.

#	You asked	What came out
1	Research HAMi in depth and find gaps for a unique CNCF project	How HAMi works (webhook, scheduler extender, device plugin, HAMi-core) and the 2026 landscape: DRA stable in 1.34, consumable capacity, CUDA 13.4 memory limits, NVIDIA open-sourcing Run:ai, KAI Scheduler, Grove and llm-d joining CNCF. Four gap ideas: GPU QoS and interference-aware co-location (top pick), closed-loop GPU rightsizing, GPU checkpoint and live migration, and a vendor-neutral GPU cluster simulator. Ideas to avoid: another fractional-GPU mechanism, another batch scheduler, multi-cluster brokering, unified-memory oversubscription
2	Build something unique on GPU QoS co-location	The **Tandem** design doc: SLO-driven QoS classes (LatencySensitive, Burstable, BestEffort), a control loop that throttles, then suspends, then evicts best-effort work, and `GPUServiceLevel` and `GPUQoSPolicy` APIs. Closest existing things: HAMi's two-level priority, Tencent qGPU and Valve
3	Explain the problem with a real example	A retailer's chatbot on one H100 with a 300 ms p99 time-to-first-token target, next to a fine-tune job. Every fixed split is wrong at some hour of the day. Valve shows the real version: 8,054 GPUs, 34.6% higher utilization
4	Find a virtualization-level problem in CNCF issue trackers that nobody in open source solves	Memory bugs are crowded and being fixed. Compute isolation is broken everywhere (HAMi #1445, HAMi-core #350 and #348, HAMi #3156). The opening: let the GPU preempt and decide from user space. Working name **Quanta**, with Tandem as a later policy layer on top
5	(Voice) Pick one domain to go deep in	Compute isolation on shared GPUs. You turned it into the goal prompt

#	You asked	What came out
6	Validate it, explain GPU scheduling, design a spike, plan integration, give a reading list	The **Quanta** design doc. No fatal flaw, but the mechanism already works in RETARGET (open research code) and Valve (closed), so the honest claim is the first production-grade, Kubernetes-native open implementation. Five limits, a 3-week spike with 12 experiments, and a 15-item reading list
7	What problem does Quanta actually solve?	The 70/30-becomes-50/50 example with evidence, saved as the doc **Quanta: The Problem It Solves**
8	Is HAMi's memory enforcement a different problem?	Half right: memory is a different problem and not Quanta's; compute is the same problem, enforced at a different point
9	(Voice) What "taking turns" means, memory, and how this fits with HAMi	The cook analogy; Quanta is weighted time-slicing; memory can't take turns; Quanta runs beside existing memory limits; HAMi-core's compute limiter gets switched off; a generic design of node agent, any memory limiter, and adapters
10	Everyone sizes GPU sharing by memory; how do we do it by time?	Copy Kubernetes' CPU model: a compute request and limit, a scheduler check that requests add up to 100% or less, node enforcement. Saved as a spreadsheet with a fit-check calculator
11	Advantages and disadvantages of time-slicing	Six of each, plus a comparison with MPS and MIG
12	Name it DynaGPU and check it's unique	No exact match, but risky near-matches: Dynamia, Dynatrace's DynaKube and NVIDIA Dynamo
13	Write full documentation	The **DynaGPU** design doc, with three diagrams
14	Can one company read another's VRAM by noting addresses?	No, because of per-process page tables, but leftovers, Rowhammer, side channels and driver bugs are real risks
15	Is any company doing GPU time-slicing?	Open-source tools only give equal turns; every weighted version is closed

#	You asked	What came out
16	Can DynaGPU stand alone on DRA, without HAMi or KAI?	Yes: DRA does the placement math, and DynaGPU adds the missing runtime enforcement

The problem DynaGPU solves

On Kubernetes today, pods sharing an NVIDIA GPU can't get unequal shares of GPU time that actually hold, while also keeping one pod's crash away from the others. Nothing open source does both.

The example used throughout: pod A is a customer-facing chatbot on vLLM, and pod B is a nightly batch job, also on vLLM. You set `nvidia.com/gpucores: 70` on A and `30` on B. When both are busy, the GPU still gives them roughly equal turns, so A gets about half the GPU and the chatbot slows down every night.

The evidence

- **CUDA graphs skip HAMi's limiter.** The limiter runs only in `cuLaunchKernel`, `cuLaunchKernelEx` and `cuLaunchCooperativeKernel`; `cuGraphLaunch` passes straight through (graph.c, line 291). vLLM and SGLang launch most decode work as CUDA graphs. Reported in HAMi #1445, still open.
- **The limiter stays off under the default policy.** A 40% limit still ran at 112 TFLOPS on an A40; only forcing it on at 10% slowed it, to 42.5 TFLOPS (HAMi-core #350). On GB10, a job took about 9.0 s at 100%, 50% and 25%, and a 75/25 pair finished in 15.8 s and 20.5 s (HAMi #3156).
- **Even when forced on, it's an estimate.** A pod forced to 25% still ran at about 46% speed (#3156), and the limit moves in coarse steps (HAMi-core #348).
- **Every launch pays for the check:** 4.2 µs without HAMi-core, 15.3 µs with it (GPU-Virt-Bench).
- **NVIDIA's open tools don't fill the gap.** KAI Scheduler and the GPU Fractioning Operator size a slice by memory only. For compute you pick equal turns (`time-slicing`) or MPS (`sm-sharing`), where all pods share one error domain (NVIDIA).
- **Closed products already sell the fix:** NVIDIA vGPU (VMs only), NVIDIA Run:ai's Strict and Fair modes, Alibaba cGPU, Tencent qGPU, Baidu's GPU Manager and Valve.

Why HAMi's design can't fully fix it: memory is a request you can refuse, but compute is time. A launch counter decides before the work runs and can't stop it once it's running. The #350 bug can be fixed and CUDA graphs can be estimated, but that last limit comes with the approach.

Who is affected: teams that put latency-sensitive inference next to batch or training work, GPU clouds that sell "30% of a GPU", and shared internal clusters that need priorities between teams.

How DynaGPU works

DynaGPU treats GPU time the way Kubernetes treats CPU time: each pod gets a request, an optional limit and a priority, and the GPU's own scheduler enforces them. It is the GPU version of Linux's `cpu.weight` and `cpu.max`.

The mechanism. The GPU already takes turns between channel groups, one per process, about 1–2 ms each, and since Pascal it can pause a process in the middle of a kernel. The driver exposes controls for those turns as ioctls on `/dev/nvidiactl`: set a group's turn length (`NVA06C_CTRL_CMD_SET_TIMESLICE`, `0xa06c0103`), take a group out of the rotation or put it back (`NVA06C_CTRL_CMD_GPFIFO_SCHEDULE`, `0xa06c0101`), preempt it now (`NVA06C_CTRL_CMD_PREEMPT`, `0xa06c0105`), or disable up to 64 channels at once (`NV2080_CTRL_CMD_FIFO_DISABLE_CHANNELS`, `0x2080110b`). No driver change is needed, but these are not a promised public API.

What DynaGPU does with them

- **Weights:** each pod's turn is its share of a target cycle: turn = cycle × request ÷ sum of requests. Requests of 70 and 30 with a 10 ms cycle give 7 ms and 3 ms turns. The GPU skips idle pods, so idle time is never wasted.
- **Caps:** a pod with a limit gets a budget per window (30% of 100 ms is 30 ms). Once it's spent, the pod leaves the rotation until the next window.
- **Preemption:** when a higher-priority pod has work, lower-priority pods are paused at once or given short turns. Priority never pushes a pod below its request over a window.
- **What this buys:** shares hold for CUDA graphs and long kernels, launches go straight to the GPU with no per-launch cost, and each pod keeps its own CUDA context, so crashes stay contained.

Finding and controlling each pod's channel group. A small shim records the handles when CUDA starts, following Valve's method of watching CUDA's startup ioctls. Three ways to send the controls: the node agent acts directly (best, but expected to fail because the driver ties handles to the process that created them), share-then-duplicate (unverified), or a control thread inside each pod (known to work, as in RETARGET, but a pod could tamper with it).

The API

Field	Meaning
Compute request (0–100)	Guaranteed share while others are busy. The scheduler keeps the sum per GPU at or below 100. Defaults to the pod's memory share
Compute limit (optional)	Hard cap per window. Unset means the pod may use idle time
Priority (`high`, `normal`, `low`)	Who preempts whom when a pod wakes up

Starting settings for the node agent, to be tuned in the spike: a 10 ms target cycle, 1 ms minimum turn, 100 ms cap window and 1 s lease. It exports per-pod GPU time, throttled time, preemptions and time spent waiting for a turn.

Worked example (from the spreadsheet): one 80 GB GPU. The chatbot asks for 50 GB and a 70% request with no limit; the batch job asks for 30 GB, a 30% request and a 30% limit. Both checks pass: 50 + 30 ⩽ 80 GB, and 70 + 30 ⩽ 100%. On the node, the memory limiter enforces memory, and DynaGPU sets 7:3 turns and caps the batch job at 30 ms per 100 ms.

Memory is not DynaGPU's job. HAMi-core or CUDA 13.4's driver limits handle it. When DynaGPU pauses a pod, its memory just stays reserved, so nothing crashes.

Two design choices: use the memory share as the default time share (as Run:ai does) but let users change it, because an LLM server holds lots of KV-cache memory even when idle; and keep turns short, because Run:ai's default 250 ms turns in a 5-second cycle make workloads less responsive.

Fail-open: if the agent dies or the driver changes, the GPU falls back to NVIDIA's default equal turns, and a paused pod resumes itself when its lease runs out.

Concepts explained along the way

These are the questions you asked as the design took shape, with the short answers.

- **What does "the GPU takes turns" mean?** Think of one cook with two customers. The GPU runs pod A's work for 1–2 ms, pauses it and saves its state, runs B, then goes back to A, hundreds of times a second. This happens inside the GPU even without HAMi. Every turn is the same length by default, which is why 70/30 comes out near 50/50.
- **Isn't that just time-slicing?** Yes, it's the same mechanism NVIDIA calls time-slicing. DynaGPU is weighted time-slicing: you choose each pod's turn length, and it adds caps and instant preemption.

- **Why does HAMi's in-container check work for memory but not compute?** Memory is a request you can refuse: the pod asks for bytes, HAMi says no, and nothing is used. Compute is time: HAMi can only guess at launch, can't pull back a running kernel, and CUDA graphs skip the check.
- **Can the same trick isolate memory?** No. Memory is space, so a paused pod's data must stay somewhere, and moving gigabytes on every turn would be far too slow. Memory limits already exist in HAMi-core and CUDA 13.4. Taking memory back from a running batch job is the hard, unsolved part (Valve and kvcached work on it) and would be a separate project.
- **If DynaGPU only handles compute, won't memory crash?** No. Memory limits add up to no more than the GPU's memory, for example 50 GB plus 30 GB on an 80 GB card. A pod that goes over gets an out-of-memory error and its neighbors keep running. Pausing a pod frees and moves nothing. Linux works the same way, with separate CPU and memory controllers used together.
- **So we need two tools?** HAMi-core for memory and DynaGPU for compute, but users see one thing. HAMi-core's compute throttle is switched off by a setting it reads once at start, so the per-launch slowdown disappears.
- **How do we make it generic?** A standalone node agent that controls GPU time, memory limits from whatever the cluster already uses (HAMi-core, CUDA 13.4 or MIG), and small adapters for HAMi, KAI Scheduler or DRA.
- **Why does everyone size GPU sharing by memory, not time?** Memory is a hard limit: if it doesn't fit, the pod crashes, and bytes are easy to count and enforce. Time is soft, since too little only slows a pod, and it's hard to enforce, so schedulers leave it to equal turns.
- **How do we share by time, then?** Copy Kubernetes' CPU model:

Part	CPU today	GPU time with DynaGPU
What the pod asks for	`requests.cpu` and `limits.cpu`	A compute request and a compute limit, separate from memory
Scheduler	Places pods so requests fit on the node	Places pods so compute requests on each GPU add up to 100% or less
Node	The kernel turns requests into weights and limits into a quota per 100 ms	DynaGPU turns requests into turn lengths and limits into a budget per window

Time-slicing: advantages and disadvantages

Time-slicing gives up some throughput in exchange for crash isolation and flexibility. Two of its costs come with the method itself; the rest are smaller or fixable.

Disadvantages

1. **Only one pod runs at a time.** During its turn a pod has the whole GPU, even if it uses only part of it, so two small pods that could run side by side under MPS take turns instead. This is the biggest cost.
2. **Each pod slows down while others are busy.** At a 50% share, 20 ms of work takes about 40 ms, and a waking request waits for the current turn.
3. **Switching costs time:** about 100 µs per switch on an A100 and 66 µs on an RTX 4000, roughly 5–10% of GPU time with 1–2 ms turns on an A100.
4. **Memory doesn't take turns.** A paused pod keeps all its memory, so every pod must fit at once.
5. **Multi-GPU jobs stall** when one part of a job is paused while the others wait for it.
6. **It isn't a security boundary.** Pods share the driver and the GPU's caches.

Two more apply to DynaGPU's approach: the driver controls are undocumented and can change with a driver update, and caps across 8 GPUs took over 5 ms in Valve without a small driver patch.

Advantages

1. **Crash isolation:** each pod keeps its own CUDA context. With MPS, one crash can take down every pod.
2. **Runs on any NVIDIA GPU,** including L4, L40S, A10 and consumer cards.
3. **Flexible:** any number of pods, any share size, changed while pods run.
4. **Nothing is wasted when others are idle:** a pod running alone gets the whole GPU.
5. **No app changes and no extra server process** on each node.
6. **The hardware enforces it,** so turns hold for CUDA graphs and long kernels.

	Time-slicing	MPS	MIG
Pods run side by side	No, they take turns	Yes	Yes, in fixed slices
One pod's crash hits others	No	Yes	No
Unequal shares	Not in open source today	Yes, by % of the GPU's cores	Yes, in fixed sizes

	Time-slicing	MPS	MIG
Works on L4, L40S, A10	Yes	Yes	No
Security boundary	No	No	Stronger

What DynaGPU changes: it can't remove disadvantages 1, 3, 4 and 6. It removes the equal-turns-only limit and shrinks disadvantage 2 for the pod that matters, because with instant preemption the chatbot waits under a millisecond (Valve's measurement) instead of a whole turn. Pitch it for bursty or mixed-priority pods that must not crash each other; MPS is still the better fit for many small, steady pods from one trusted team.

Security: can tenants read each other's VRAM?

No, not by remembering addresses, but time-slicing between two companies is still not safe. You framed it as a shared room with a number lock: one person learns the code and walks in during the other's turn. A GPU doesn't work like that.

Why the room analogy doesn't hold

- **The rooms don't take turns.** VRAM is a building with many rooms. Each company keeps its own rooms the whole time; only the workbench (the GPU cores) is shared in turns.
- **There's no code to learn.** Every pointer is a virtual address. The GPU's memory management unit (MMU) translates it through that process's own page tables, which only the driver writes. If A and B both ask for "room 42", the guard sends each to a different physical room.
- **Each turn switches page tables.** Each channel group points to its own page tables, so a switch of turns is also a switch of lists.
- **Trying someone else's room gets you thrown out.** An address outside your page tables causes a page fault (Xid 31) that kills only your context. User code has no way to read a physical address directly. NVIDIA's MPS docs say even MPS clients each "own [their] own GPU address space" (MPS docs).

The real risks

1. **Leftovers after checkout.** NVIDIA's docs say `cudaMalloc` memory "is not cleared", and NVIDIA doesn't promise to clear memory between processes. Papers from 2014 to 2024 recovered other processes' leftovers in some setups, with no confirmed real-world breach (research summary).
2. **On-chip leftovers.** LeftoverLocals (2024) leaked data on AMD, Apple and Qualcomm GPUs; NVIDIA confirmed it was not affected.

3. **Rowhammer: changing the guard's list.** GPUBreach (2026) flipped bits in GDDR6 to corrupt GPU page tables and, with a driver bug, got root on the host even with the IOMMU on. GPUThor (Aug 2026) defeated ECC on RTX A6000, A5000, A4500 and A4000; A10, L4, L40, RTX 4090 and A30 showed no flips, and A100 and H100 weren't tested. This is the real-world version of your worry.
4. **Listening through the wall:** timing side channels can reveal patterns, such as a model's shape.
5. **Driver bugs:** both tenants share one kernel driver.

How it's solved: don't time-slice a GPU between companies that don't trust each other. Use MIG, whose instances have "separate and isolated paths through the entire memory system" (MIG guide), or a whole GPU per company in separate VMs with a reset between tenants, and keep ECC on and drivers patched. Within one company, the MMU is enough, which is why DynaGPU targets trusted multi-tenancy.

Who else works on GPU time-slicing

The demand is proven, but every weighted or priority-based version is closed, tied to one cloud, or VM-only; the open-source tools give only equal turns.

Who	What they offer	Open?
NVIDIA (device plugin, GPU Fractioning Operator, KAI Scheduler)	Equal turns, or MPS; no weights or caps	Yes
Google GKE	GPU time-sharing: "every process gets a fair timeslice"; no memory limits	Managed service
NVIDIA Run:ai	Strict and Fair time-slicing: time share equals the GPU fraction; 250 ms turns by default	No; the open-source release replaced this part
NVIDIA vGPU	Equal-share and fixed-share schedulers	No, VMs only
Alibaba Cloud	Six cGPU policies, including fixed time slices and weight-based preemption	No

Who	What they offer	Open?
Tencent Cloud	qGPU online/offline deployment with suspend and resume; Valve research on 8,054 GPUs	No
Baidu Cloud	CCE GPU Manager pauses offline tasks during online peaks	No
Midokura	Weighted 100 ms windows inside gVisor's GPU proxy	No repo linked
Zesty	Automates memory allocation and placement on NVIDIA's open-source fractioning (Oct 15, 2026); no weighted turns	No

Research code: RETARGET sets per-process timeslices from user space but is a one-commit artifact with no license or Kubernetes. GPREEMPT needs a patched driver. gpu_ext modifies the open driver. SPRIO offers fixed priorities only, and LithOS builds on MPS. A 2025 position paper calls driver-level dynamic time-slicing future work.

NVIDIA's "time-based fairshare" in KAI is about fairness between team queues over hours, not GPU turns.

The biggest competitive risk is NVIDIA itself. It already has weighted time-slicing in commercial Run:ai and could move it into the open GPU Fractioning Operator, which is why the plan is to integrate and upstream early.

Design direction: building DynaGPU on DRA alone

You decided you want DynaGPU as its own project built on DRA, not on HAMi or KAI Scheduler. That works, and DRA is the cleanest home: DRA already does the placement math, and DynaGPU adds the runtime enforcement it lacks.

- **What DRA gives you:** with consumable capacity, many claims share one GPU and kube-scheduler adds up each claim's slice. It is beta and on by default from Kubernetes 1.36, with GA targeted for 1.38 (KEP-5075). A GPU publishing `compute: 100` can't be promised more than 100, so no custom scheduler is needed.
- **What's missing:** "capacity is scheduling accounting, not runtime isolation" (NVIDIA DRA driver #1609); runtime enforcement is "out of scope by design" for DRA (CNCF blog).

Two ways to build it

1. **An enforcer for any DRA driver (recommended).** DynaGPU doesn't own the GPUs; it enforces the compute share the driver allocated. Small scope, vendor-neutral, and complementary to NVIDIA's and HAMi's drivers: "the runtime half of DRA for GPU compute."
2. **Your own DRA driver.** Full control, but you must handle discovery, health and device injection yourself, it overlaps NVIDIA's and HAMi's drivers, and only one driver can own a given GPU.

How the enforcer works: the driver publishes each GPU as shareable with `memory` and `compute` capacity. A pod's claim asks for a slice of each (`spec.devices.requests[].exactly.capacity.requests`). kube-scheduler places it only where both fit. The DynaGPU node agent watches claims allocated on its node, reads the compute each one consumed, injects the shim through NRI and sets turn lengths. Limit and priority come from annotations until a standard field exists.

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaimTemplate
metadata:
  name: chatbot-gpu
spec:
  spec:
    devices:
      requests:
        - name: gpu
          exactly:
            deviceClassName: gpu.example.com   # set by the driver you use
            capacity:
              requests:
                memory: 50Gi
                compute: "70"   # the driver must publish this capacity
```

The catches

- **A driver must publish compute.** HAMi's DRA driver already publishes memory and compute. NVIDIA's offers only `memory` and an alpha `shares` capacity (guide), so either treat `shares` as the compute weight or propose a standard `compute` capacity upstream.
- **Memory comes from elsewhere:** CUDA 13.4 per-container caps, which NVIDIA's DRA driver plans to use (driver r615 or newer, cgroup v2), or HAMi-core.
- **Kubernetes 1.36 or newer** and a container runtime with NRI.

- **Talk to the HAMi maintainers early.** Their DRA driver already publishes compute and enforces it with HAMi-core, so DynaGPU fits as an alternative enforcer for it, not a rival.

With DRA, the "scheduler adapter" box in the design doc disappears, because kube-scheduler does that job. The design doc still describes the earlier plan: adapters for HAMi (reuse `gpucores`), KAI (add `gpu-compute.request` and `gpu-compute.limit`) and DRA.

Name and CNCF listing

Nobody uses the exact name DynaGPU as far as I could check, but I advised against it because it sits close to Dynamia, the company behind HAMi.

Exact matches checked

- **CNCF landscape:** no entry in the landscape data as of Nov 6, 2026.
- **Package registries:** nothing on PyPI, npm or crates.io.
- **GitHub:** no public repos at `dynagpu/dynagpu` or `dyna-gpu/dyna-gpu`. A full GitHub search wasn't possible without a linked GitHub account.
- **Only exact hit:** a `dynagpu` source folder in an old UT Austin molecular-dynamics code. Not a product, so no conflict.

Near-matches that cause trouble

- **Dynamia** sells "enterprise GPU virtualization and scheduling, built on the CNCF project HAMi" and became a CNCF member in September 2026. As a HAMi contributor, people may assume DynaGPU is their product. This is the biggest risk.
- **Dynatrace**, a CNCF member since 2017, configures its Kubernetes operator through a custom resource called `DynaKube`.
- **NVIDIA Dynamo**, NVIDIA's inference framework for Kubernetes; searching "Dyna GPU" already returns its pages.
- **Positioning:** "Dyna" suggests dynamic allocation, the crowded area you wanted to avoid; the project is really about sharing GPU time.
- **The earlier working name, Quanta,** clashes with Quanta Computer and its QCT brand, a major GPU server maker.

Not checked from here: domains, Docker Hub and trademark databases, which this environment blocks. Check a registrar (.io, .dev, .com), Docker Hub, the GitHub organization name, and the USPTO and WIPO trademark tools before announcing.

CNCF notes

- The landscape generally includes cloud-native projects with at least 300 GitHub stars that fit an existing category.

- When a project joins CNCF, "the project name is transferred to the Linux Foundation" (CNCF Sandbox: Know Before You Submit), so a name close to two CNCF members' brands could make that step harder.
- Apply to the CNCF Sandbox once the project has users outside the core team.

Risks and the validation plan

The verdict so far is that there's no fatal flaw, but a three-week spike on one GPU must prove the mechanism before anything else, and week 1 alone can stop the project.

The five accepted limits from the Quanta review

1. **No concurrency between pods:** an engine runs one context at a time.
2. **Not a security boundary:** if control runs inside each pod, a hostile pod could raise its own turn length.
3. **Caps are harder than weights:** they need accurate per-pod GPU time and fast control; Valve saw over 5 ms on an 8-GPU node until it patched one driver line.
4. **Private interface:** the controls are in NVIDIA's open driver headers but are not a promised API.
5. **NVIDIA owns the knob:** it could add weights to its own operator, so stay vendor-neutral and upstream early.

The spike: one A100, L4 or H100 on NVIDIA's open kernel modules, with nvdebug, Nsight Systems, NVML and `dmesg` as ground truth, a small `ioctl`-wrapping probe library, and six workloads, from a plain kernel loop to vLLM with CUDA graphs and a 2-rank NCCL loop.

#	Question	Pass if
E1	Can we find every context's channel group?	Every context maps to its group, for PyTorch and vLLM
E2	Which control route works?	The in-process route works; record what the other two return
E3	Do turn lengths give weighted shares?	Split within ±5% of target up to 9:1
E4	Is it work-conserving?	The busy pod gets at least 95% of solo throughput while the other idles
E5	Do shares hold with CUDA graphs and long kernels?	Within ±5% where HAMi-core's limiter fails

#	Question	Pass if
E6	How fast is preemption?	Added p99 start delay of 1 ms or less with pausing
E7	What does it cost?	At most 5% loss at 1 ms turns or longer; no launch-latency change
E8	Does work resume correctly?	10,000 pause cycles with identical outputs, no Xid errors, no hangs
E9	Can we enforce a hard cap?	Within 3 points of the cap
E10	What if the agent dies?	Each pod resumes itself when its 1 s lease expires
E11	Does NCCL survive?	No errors; pausing both ranks stops the peer from spinning
E12	How stable is the interface?	Control IDs and structures unchanged from driver 535 to 595, or a small per-version table

Decision rules: stop after week 1 if no control route works or E3 misses ±5%; stop after week 2 if pausing ever corrupts output (E8); go if E1–E8 and E10 pass, with E9 and E11 failures documented as limits.

Open questions still to answer: which control route works; how the agent learns that a high-priority pod has new work without intercepting launches; which source measures per-pod GPU time precisely enough for caps; whether the controls behave the same on NVIDIA's proprietary driver and inside a MIG slice; and whether NVIDIA will support these controls or add weights itself.

Everything created in this conversation

Four docs, one spreadsheet and three inline pictures, plus this summary.

Item	What it holds
Tandem: SLO-Driven GPU QoS for Kubernetes	The first design: SLO-driven GPU QoS classes, the control loop, `GPUServiceLevel` and `GPUQoSPolicy` APIs, architecture, roadmap and benchmark plan. Now a possible policy layer on top of DynaGPU

Item	What it holds
Quanta: Preemptive GPU Time-Sharing	The deep-dive: verdict and five limits, evidence tables, how NVIDIA GPU scheduling works, the driver controls, the 3-week spike, integration plan and a 15-item reading list
Quanta: The Problem It Solves	The plain-language problem: the 70/30 example, why HAMi's limit doesn't hold, NVIDIA's options, and how turn-taking fixes it, with a picture
DynaGPU: Weighted GPU Time-Sharing for Kubernetes	The full design doc: problem, solution, comparison with 13 alternatives, architecture, scheduling internals, driver controls, API, logic, failure handling, integration, risks, validation plan and open questions, with three diagrams
`Quanta_GPU_time_sharing.xlsx` (sent in chat)	Seven tabs: who sizes GPU sharing by what, memory vs time, the CPU model, a fit-check calculator, design choices, plug-in points and sources
Inline pictures in chat	The chatbot's GPU need across a day against a fixed 50% cap (illustrative); GPU turns today vs weighted vs capped; and the "same room number, different rooms" MMU picture

Where things stand

The idea is settled and documented; the next real step is the week-1 spike, and the name is still open.

Decided

- **One domain:** GPU time-sharing, meaning compute isolation between pods on a shared NVIDIA GPU.
- **Mechanism:** the GPU's own turn-taking, controlled through driver calls, never per-launch interception.
- **Scope:** compute only; memory stays with HAMi-core or CUDA 13.4.
- **Trust model:** teams inside one company. Separate companies get MIG slices or separate GPUs.

- **Direction:** a standalone, DRA-native project, preferably as an enforcer that works with any DRA driver.

Open decisions

- [ ] Pick the final name, then check domains, Docker Hub, the GitHub organization and trademarks.
- [ ] Choose between an enforcer for any DRA driver and your own DRA driver.
- [ ] Run week 1 of the spike (E1–E4), which decides go or no-go.
- [ ] Talk to the HAMi maintainers, whose DRA driver already publishes compute. Your HAMi-core PR #353 touches the limiter code involved, if that PR is yours.
- [ ] For NVIDIA's DRA driver, map its `shares` capacity to compute weights or propose a standard `compute` capacity upstream.

Offers still open from me

- Update the DynaGPU design doc to the DRA-only design.
- Add a memory-isolation note to the design doc's security section.
- Suggest five names that point at GPU time and run the same checks on each.
- Pull full issue counts from the trackers once you link a GitHub account.
- Add the real-world example to the Tandem doc.

Earlier offers from the Tandem stage, such as a HAMi-core proposal to change the compute limit while a pod runs, were overtaken by the move to DynaGPU.